

```
!pip install pygad numpy
```

```
Collecting pygad
  Downloading pygad-3.7.0-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.13/dist-packages (from pygad) (3.1.2)
Downloading pygad-3.7.0-py3-none-any.whl (218 kB)
----- 218.3/218.3 kB 5.7 MB/s eta 0:00:00
Installing collected packages: pygad
Successfully installed pygad-3.7.0
```

```
import numpy as np
import pygad

# --- 1. System Parameters & Time Series (24 Hours) ---
np.random.seed(42)
hours = 24
load = np.array([50, 45, 42, 40, 43, 55, 70, 85, 90, 85, 80, 75,
                 72, 70, 75, 85, 100, 110, 105, 95, 85, 75, 65, 55]) # kW

solar_gen = np.array([0, 0, 0, 0, 5, 20, 45, 70, 90, 100, 105, 110,
                     105, 95, 80, 60, 35, 10, 0, 0, 0, 0, 0, 0]) # kW

wind_gen = np.array([30, 35, 40, 35, 30, 25, 20, 15, 20, 25, 30, 35,
                    40, 45, 40, 35, 30, 25, 30, 35, 40, 45, 35, 30]) # kW

grid_import_price = np.array([0.10]*6 + [0.20]*12 + [0.15]*6) # $/kWh per hour
grid_export_price = 0.05 # $/kWh

battery_capacity = 100.0 # kWh
max_charge_discharge = 30.0 # kW

# Decision variables per hour: Battery Power (P_batt > 0: charging, P_batt < 0: discharging)
# Total variables = 24

def fitness_func(ga_instance, solution, solution_idx):
    # solution contains 24 values representing battery action per hour (-30 to 30 kW)
    p_batt = solution
    soc = 50.0 # Initial State of Charge in kWh
    total_cost = 0.0
    penalty = 0.0

    for t in range(hours):
        # Update SOC
        # Charging efficiency 90%, discharging efficiency 90%
        if p_batt[t] >= 0:
            soc += p_batt[t] * 0.9
        else:
            soc += p_batt[t] / 0.9

        # Check battery limits
        if soc < 10.0 or soc > battery_capacity:
            penalty += 1000.0 # Severe penalty for violating bounds

        # Net power balance: Generation + Battery_discharge(-p_batt) + Grid = Load
        # Note: if p_batt is negative, it discharges, adding power to system
        available_gen = solar_gen[t] + wind_gen[t] - p_batt[t]
        grid_power = load[t] - available_gen

        if grid_power > 0:
            # Buying from grid
            total_cost += grid_power * grid_import_price[t]
        else:
            # Selling excess to grid
            total_cost += grid_power * grid_export_price # negative cost (revenue)

    # We want to minimize cost, so fitness is the inverse of (cost + penalty)
    fitness = 1.0 / (total_cost + penalty + 1e-5)
    return fitness

# --- 2. Genetic Algorithm Configuration ---
gene_space = {'low': -max_charge_discharge, 'high': max_charge_discharge}

ga_instance = pygad.GA(
    num_generations=100,
    num_parents_mating=20,
    fitness_func=fitness_func,
    sol_per_pop=50,
    num_genes=hours,
    gene_space=gene_space,
```

```
mutation_percent_genes=15,  
parent_selection_type="sss",  
crossover_type="single_point"  
)  
  
# --- 3. Run Optimization ---  
ga_instance.run()  
  
solution, solution_fitness, solution_idx = ga_instance.best_solution()  
print(f"Optimal Battery Schedule (kW per hour): \n{np.round(solution, 2)}")  
print(f"Best Fitness Value: {solution_fitness}")
```

```
Optimal Battery Schedule (kW per hour):  
[ -4.85 -13.82 -8.92 17.59  4.47 -4.67 -5.82 -12.57 16.45  9.11  
  8.48 17.6  18.71  9.4  -0.62  4.14 -24.14 -19.98 -8.63  7.7  
  6.13  1.92 -19.34 -9.14]  
Best Fitness Value: 0.025585565634364246
```